

Sungjun Lee

Vancouver, BC, Canada | 778-926-8205 | anderpuding0917@gmail.com | github.com/anderpudding | Portfolio

Professional Profile

Computer Science and Mathematics student with experience in full-stack software engineering, native iOS development, AI-powered recommendation systems, quantitative optimization, and research data analysis. Developed and deployed applications using Next.js, React, SwiftUI, Supabase, PostgreSQL, OpenAI, and Gemini, with particular experience in REST API integration, authentication, asynchronous workflows, mobile media handling, database security, and production debugging. Additional technical interests include numerical optimization, graph algorithms, Monte Carlo simulation, statistical modeling, and explainable recommendation systems.

Education

University of British Columbia (UBC)

Vancouver, BC, Canada

Bachelor of Science, Combined Major in Computer Science and Mathematics

Expected May 2027

Selected Computer Science Coursework: Systematic Program Design, Models of Computation, Software Construction, Computer Systems, Data Structures and Algorithms, Software Engineering, Numerical Computation, Artificial Intelligence, Computer Hardware and Operating Systems

Selected Mathematics Coursework: Advanced Calculus I–IV, Vector Calculus, Differential Equations, Probability Theory, Optimization, Linear Programming, Applied Linear Algebra, and Statistical Probability

Professional Experience

Product Manager Accelerator

Remote

Software Engineer Intern – AI/ML

Apr. 2026–Present

Internship expected to conclude in Aug. 2026

Product: Remember – AI-Powered Memorial Platform

Next.js, React, TypeScript, Node.js, Express, Supabase, PostgreSQL, OpenAI, Tailwind CSS, Framer Motion

- Developed and deployed a full-stack memorial platform that collects personal stories, photographs, questionnaire responses, and voice recordings from invited contributors and transforms the submitted material into interactive AI-generated memorial experiences.
- Built approximately 15–20 new application pages and modified approximately 50 existing pages while aligning the contributor, organizer, and memorial-output experiences with evolving product requirements, backend contracts, and design specifications.
- Initially implemented selected portions of the contributor experience and later assumed end-to-end ownership of the complete flow, including invitation validation, contributor registration, relationship selection, questionnaire completion, photo upload, voice upload, review, submission, and confirmation.
- Integrated the frontend with 29 unique backend API endpoints, including 26 active endpoints used in the main application. Implemented or substantially contributed to 17 endpoint integrations covering contributor onboarding, draft persistence, media uploads, submission, organizer review, signed media access, and AI-generation status polling.
- Improved workflow reliability by introducing 30-second request timeouts, up to three automatic retries for rate-limited autosave requests, a 60-second invitation-validation cache, explicit autosave status indicators, local draft recovery, and user-facing error and recovery states.

- Expanded mobile media compatibility by supporting HEIC and HEIF photographs and M4A, MP3, WAV, and WebM voice recordings, addressing format differences between desktop browsers, Android devices, and iPhones.
- Developed responsive Story and Voices experiences with animated transitions, slideshow navigation, contributor context, expandable transcripts, audio scrubbing, elapsed and remaining time displays, and a reusable custom audio player.
- Implemented organizer-facing contribution management for reviewing submissions, updating approval status, removing contributions, and coordinating cleanup of associated database and object-storage records.
- Investigated and resolved frontend-backend contract mismatches, authorization failures, rate limits, upload errors, unavailable media, AI-generation failures, environment configuration issues, and deployment differences between local and hosted environments.
- Collaborated with five software developers and two data scientists as part of a broader 15-person product team spanning engineering, AI, product management, and design.
- Supported validation of the deployed MVP through internal testing by the 15-person team and approximately 20 external testers. The application was deployed using Vercel for the frontend and Render for backend services.
- Contributed more than 100 Git commits and authored at least 17 merged pull requests during MVP development, feature integration, product refinement, and launch preparation.

Research Experience

Research Assistant

PLACE Lab – Planning for Climate Equity, University of British Columbia

- Developed reusable Python data-processing and visualization workflows for examining relationships between climate exposure, demographic characteristics, geographic conditions, and regional equity indicators.
- Cleaned, transformed, joined, and reorganized raw research datasets into structured analytical formats suitable for comparative, longitudinal, and geographic analysis.
- Produced analytical visualizations using Matplotlib and Plotly, including comparative charts and region-level displays designed to make complex climate and demographic patterns interpretable to researchers from multiple disciplines.
- Structured data and visual outputs to support investigation of unequal climate impacts, community vulnerability, urban planning priorities, and potential policy interventions.
- Improved research reproducibility by separating data preparation, transformation, analytical calculations, and visualization into reusable Python components rather than relying on manually generated figures.
- Collaborated with interdisciplinary researchers to translate quantitative findings into clear research narratives and policy-relevant interpretations while maintaining appropriate caution around data limitations and causal claims.

Technical Projects

Vanori – AI-Powered Local Discovery iOS Application

Apr. 2026–Present

Swift, SwiftUI, Supabase, PostgreSQL, Deno Edge Functions, Gemini 2.5 Flash-Lite, Google Places

- Developed a native iOS application that helps Vancouver residents discover personalized food, event, and lifestyle recommendations based on location, budget, interests, preferred atmosphere, language preferences, and user life stage.
- Built approximately 40 application screens spanning authentication, onboarding, user-preference collection, personalized recommendations, category discovery, nearby content, content details, bookmarks, reviews, notifications, community functionality, and profile management.

- Structured the application using an MVVM-style architecture with SwiftUI views, `ObservableObject` view models, service abstractions, Swift concurrency, and local persistence for user preferences, cached content, and saved-state recovery.
- Designed a hybrid recommendation architecture that first removes unsuitable content through pre-filtering and then scores eligible content across seven components: category match, vibe match, location match, budget match, content quality, engagement, and freshness or diversity.
- Defined different recommendation-weight profiles for students, working-holiday residents, workers, long-term residents, and baseline users, allowing the relative importance of affordability, proximity, quality, atmosphere, and novelty to change by user context.
- Designed a ranking pipeline in which a Supabase Edge Function produces the top 20 deterministic candidates before Gemini 2.5 Flash-Lite reranks the top five candidates and returns structured recommendation reasons and confidence values.
- Limited the language model to selecting from known candidates rather than generating new places or events, reducing hallucination risk and ensuring that every recommendation maps to validated application content.
- Implemented or designed PostgreSQL data structures for user preferences, content items, user interactions, saved items, and reviews, with Supabase Auth, Row Level Security, object storage, and Edge Functions supporting authorization and server-side processing.
- Created a structured content model containing category, subcategory, geographic area, coordinates, price level, vibe tags, activity tags, descriptions, source metadata, quality scores, community-fit scores, engagement counts, ratings, and event timestamps.
- Defined a seven-source content strategy combining FSQ OS Places, Google Places, Meetup, Eventbrite, Luma, City of Vancouver Open Data, and internally curated content to balance coverage, licensing constraints, metadata quality, and Korean-community relevance.
- Separated permanent base data, display-only enrichment data, event sources, municipal open data, and internally curated information to avoid relying on restricted external review content for AI training or long-term independent storage.
- Populated the MVP with approximately 100 demo places, events, and activities and supported user testing before submitting the application to TestFlight.

AlgoQuantEngine – Quantitative Portfolio Optimization System

Python, NumPy, pandas, SciPy, Matplotlib, pytest

- Designed a modular quantitative-analysis engine that accepts arbitrary N -asset CSV inputs, requires a minimum of two assets for optimization, and has been benchmarked using synthetic portfolios containing up to 120 assets.
- Organized the system into eight major capability areas: data processing, mean–variance optimization, graph and clustering analysis, graph-constrained portfolio construction, Monte Carlo simulation, risk analysis, static and rolling backtesting, and reporting and visualization.
- Implemented mean–variance and minimum-variance portfolio optimization with support for weight constraints, normalization, portfolio statistics, and efficient-frontier generation.
- Constructed correlation networks to model relationships between assets and implemented Kruskal’s minimum spanning tree algorithm and spectral clustering to analyze portfolio structure and identify groups of strongly related assets.
- Developed graph-constrained portfolio functionality that combines conventional numerical optimization with structural information obtained from asset-correlation networks and clustering.
- Implemented Value at Risk, Conditional Value at Risk, maximum drawdown, Monte Carlo simulation, and historical risk calculations to evaluate portfolio behavior under multiple risk perspectives.
- Built static and rolling backtesting pipelines that compare mean–variance portfolios against equal-weight

and minimum-variance portfolios using portfolio returns, volatility, Sharpe ratio, equity curves, and drawdown behavior.

- In a committed demonstration dataset, the mean–variance portfolio produced higher expected return and Sharpe ratio, lower expected volatility, and a slightly higher net backtest return than the equal-weight portfolio. Treated these findings as example-specific results rather than evidence of general market outperformance.
- Authored 19 automated test cases across nine test files covering data loading, validation, optimization, graph analysis, hybrid constraints, risk calculations, rolling comparisons, backtesting, and reporting, with all 19 tests passing.
- Generated command-line summaries, CSV portfolio weights, efficient frontiers, benchmark comparisons, strategy results, JSON risk and constraint reports, and PNG visualizations including correlation graphs, cluster heatmaps, rolling equity curves, strategy dashboards, and drawdown charts.

Blackjack AI Engine – Strategy Optimization and Simulation Framework

Python, Expected-Value Analysis, Monte Carlo Simulation, pytest

- Developed a blackjack decision engine that evaluates 350 strategy-table states across hard totals, soft totals, pairs, and dealer upcards.
- Computed approximately 1,150 legal action expected-value combinations under the default rule configuration, evaluating the relative value of hitting, standing, doubling, splitting, and surrendering when each action is legally available.
- Designed the engine to support configurable dealer S17 and H17 behavior, doubling rules, double after split, late surrender, resplitting, split-ace behavior, maximum splits, and partial finite-shoe settings.
- Implemented hard-total, soft-total, and pair-strategy table generation together with command-line recommendations and detailed expected-value breakdowns.
- Built a configurable Monte Carlo simulation pipeline for evaluating long-run strategy behavior under different rule settings, with a default experiment size of 1,000 trials and no fixed configured maximum.
- Performed targeted validation against expected blackjack decisions and structured the project so that additional strategy-table comparisons and rule configurations can be tested independently.
- Authored nine pytest-style test cases across three test files covering recommendations, state evaluation, and strategy-table generation.
- Generated 37 committed reporting artifacts, including hard, soft, and pair strategy tables, expected-value comparisons, and rule-analysis reports in CSV, HTML, and PNG formats.

TeamTracker – Team and Player Statistics Application

Java, Swing, JUnit 5, JSON

- Designed an object-oriented team-management system for maintaining player rosters, recording individual statistics, summarizing team performance, and persisting application state.
- Developed a Java Swing graphical interface with dedicated panels for roster management, player creation, statistical entry, and team-level data visualization.
- Separated domain logic, persistence, event logging, and user interface responsibilities to improve maintainability and support independent testing.
- Implemented JSON-based save and load functionality to preserve team and player data across application sessions.
- Added JUnit tests for core model behavior, persistence operations, and expected state transitions, and used structured event logging to record important user actions.

Teaching Experience

Independent Mathematics Tutor

- Provided individualized mathematics instruction to more than 20 students ranging from Grade 1 through Grade 12, including AP Calculus and UBC MATH 100 and MATH 101.
- Designed lesson plans and progression frameworks based on each student's existing knowledge, course curriculum, assessment schedule, and long-term academic objectives.
- Taught algebra, functions, polynomial operations, trigonometry, sequences, systems of equations, probability, differential calculus, and integral calculus.
- Developed diagnostic exercises and custom practice materials to identify conceptual gaps and distinguish errors caused by calculation, notation, interpretation, or incomplete prerequisite knowledge.
- Supported students preparing for BC secondary mathematics courses, university calculus, AP examinations, and Canadian mathematics competitions.
- Currently provides Pre-Calculus 10 instruction emphasizing functional reasoning, mathematical communication, algebraic modeling, and trigonometric problem solving.

Technical Skills

Programming Languages:

Python, Java, Swift, TypeScript, JavaScript, SQL, C, C++, MATLAB

Frontend and Mobile Development:

Next.js, React, SwiftUI, Tailwind CSS, Framer Motion, HTML, SCSS, D3.js, Wavesurfer.js, Java Swing

Backend, Database, and Cloud:

Node.js, Express, Supabase Auth, Supabase Storage, PostgreSQL, Row Level Security, Supabase Edge Functions, Deno, Firestore, Firebase, REST APIs

Artificial Intelligence and Data Science:

OpenAI API, Gemini API, recommendation systems, deterministic ranking, LLM reranking, NumPy, pandas, SciPy, scikit-learn, TensorFlow

Data Visualization and Quantitative Computing:

Matplotlib, Plotly, numerical optimization, graph analysis, spectral clustering, Monte Carlo simulation, portfolio optimization, risk analysis, statistical modeling

External APIs and Data Services:

Google Places API, FSQ OS Places, Foursquare, Eventbrite, Meetup, Luma, City of Vancouver Open Data

Testing and Development Tools:

Git, GitHub, Linux, Jupyter, VS Code, Xcode, Supabase CLI, Docker, ESLint, pytest, JUnit 5

Additional Software:

Microsoft Excel, Microsoft Word, Adobe Photoshop, Adobe Illustrator, Adobe Premiere Pro, Adobe Lightroom

Technical Concepts:

Full-stack application development, object-oriented design, software architecture, asynchronous programming, API integration, relational database design, authentication and authorization, Row Level Security, media-upload workflows, recommendation systems, numerical optimization, graph algorithms, simulation, statistical analysis, and data visualization